

Feature Selection and Dimensionality Reduction

CIS 635 — Knowledge Discovery & Data Mining

Part 3 of 3

Course Reading Material

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.

For permissions or questions regarding the use of this material, contact hasaka@gvsu.edu.

Please cite [4] this material when using or reproducing it.

Unit: From Data to Discovery — Part 3: Feature Selection and Dimensionality Reduction

This is Part 3 of a three-part reading. Part 1, **Basic EDA Concepts: Statistical Foundations, Data Preparation, and Exploratory Analysis Foundations**, and Part 2, **Advanced EDA Concepts:**

Combining Data, Multivariate Techniques, and Distance Metrics, cover the statistical, preprocessing, EDA, and similarity/distance foundations this reading builds on — in particular, the curse of dimensionality (Part 2, Section 3.4), which is the single biggest reason feature selection and dimensionality reduction matter in practice.

Contents

1	Feature Selection and Dimensionality Reduction	3
1.1	Why Feature Selection Matters	3
1.2	Feature Selection versus Dimensionality Reduction	4
1.3	A Simple Intuition for Dimensionality Reduction	4
1.4	Common Feature-Selection Approaches: Filter, Wrapper, and Embedded Methods	5
1.5	Feature Selection in Practice: Leakage and Bias	6

Notation and Terminology Quick Reference

The table below collects the recurring terms this reading builds on, for quick lookup while reading or reviewing. Each term is developed fully in the section listed. Terms developed in Part 1 and Part 2 — population/sample, sampling bias, correlation, MCAR/MAR/MNAR, tidy data, data leakage, merging and joining, and the curse of dimensionality — appear in those readings' own quick-reference tables.

Term	Short definition	§
Irrelevant / redundant feature	A feature that adds no information, or that repeats information another feature already carries	1.1
Variance inflation factor (VIF)	A number flagging how redundant a feature is with the others	1.1
Dimensionality reduction	Building a smaller set of <i>new</i> features that summarize the original ones	1.2
Principal component analysis (PCA)	The most common dimensionality-reduction technique; combines correlated features into new ones	1.3
Filter / wrapper / embedded	The three families of feature-selection method	1.4

Learning Objectives

This reading assumes the statistical, preprocessing, EDA, and similarity/distance foundations covered in Parts 1 and 2. After completing this reading, you should be able to:

- Explain, with a simple example, why having more features is not automatically better, and distinguish irrelevant from redundant features.
- Explain the difference between feature selection and dimensionality reduction, and build intuition for how a technique like PCA combines correlated features into a smaller set of new ones.
- Apply filter, wrapper, and embedded feature-selection strategies, and explain the speed/accuracy/leakage-risk trade-offs among them.
- Recognize and avoid the most common way feature selection leaks information from a test set into a model.

1 Feature Selection and Dimensionality Reduction

1.1 Why Feature Selection Matters

Imagine trying to predict a house's sale price from a spreadsheet with fifty columns: square footage, number of bedrooms, year built, zip code, school district rating — and also things like the seller's favorite color, the day of the week the listing was created, and the listing agent's shoe size. Common sense says most of these fifty columns will not help predict price, and a few clearly will. **Feature selection** is the formal process of telling the two apart: identifying which subset of available features should actually be used for a given analytical question, rather than throwing every available column at a model and hoping for the best.

Not every recorded variable is useful, and having more features is not automatically better — extra, uninformative columns add noise, slow down computation, and (as Part 2, Section 3.4 showed) make distance-based methods behave worse as dimensionality grows. Two specific problems show up constantly:

- An **irrelevant** feature carries no information about the outcome or pattern of interest and mainly adds noise (the agent's shoe size).
- A **redundant** feature carries information that is already captured by another feature and adds complexity without adding anything new (square footage in square feet *and* square footage in square meters are the same information twice).

Feature selection connects directly back to EDA: correlation analysis (Part 1, Section 3.5), high-dimensional exploration (Part 2, Section 2.3), and case-specific domain knowledge about which variables are known to matter are usually what first suggest which features are relevant, redundant, or irrelevant.

Tip

A simple check for redundancy. Suppose a housing dataset includes both `square_feet` and `num_rooms`, and the two turn out to have a correlation of $r = 0.93$ with each other. That is a strong hint they are largely redundant — a model that already has `square_feet` learns very little new from also having `num_rooms`.

Redundancy among numerical features is often quantified more formally with the **variance inflation factor (VIF)**,

$$\text{VIF}_i = \frac{1}{1 - R_i^2},$$

where R_i^2 is the proportion of variance in feature i explained by a regression of feature i on all the other features; a VIF well above the common rule-of-thumb threshold of 5–10 signals that a feature is largely redundant with the others and a candidate for removal or combination. A feature with $R_i^2 = 0.9$ (90% of it is predictable from the other features) has $\text{VIF} = 1/(1 - 0.9) = 10$ — right at the threshold where most analysts start to worry.

1.2 Feature Selection versus Dimensionality Reduction

Feature selection chooses a subset of the *original* features, preserving their individual interpretability (a selected feature is still “square footage” or “systolic blood pressure”). **Dimensionality reduction** techniques instead construct new features as combinations of the original ones, typically achieving greater compression but at the cost of interpretability — a new, combined feature is a weighted mixture of several original variables and does not, by itself, correspond to a single measurable quantity. Both address the curse of dimensionality (Part 2, Section 3.4), but the choice between them often comes down to whether interpretability of the final feature set matters for the application (it typically does in clinical and policy settings, less so in a purely predictive system).

1.3 A Simple Intuition for Dimensionality Reduction

The most common dimensionality-reduction technique is **principal component analysis (PCA)**. The idea is easiest to see with two features that are highly correlated. Suppose five students have a midterm score and a quiz average:

Student	A	B	C	D	E
Midterm score	92	78	85	60	71
Quiz average	88	74	90	58	69

These two columns move together almost perfectly — a student who scores high on one tends to score high on the other, because both are really measuring the same underlying thing: how well the student has learned the material. PCA notices exactly this kind of redundancy and replaces the two original columns with one new column (the *first principal component*) that is a weighted combination of both, chosen so that it captures as much of the shared variation as possible. Going from 2 features down to 1 loses very little information here, because the two original features were telling almost the same story; the students would rank in nearly the same order using either the original two columns or the single combined score.

Key Idea

Feature selection asks “which of my *existing* columns should I keep?” Dimensionality reduction asks “can I build a smaller number of *new* columns that capture most of what the existing ones were telling me?” PCA answers the second question by finding the directions along which the data varies the most, and using those as the new, compressed feature set.

```
from sklearn.decomposition import PCA
import pandas as pd
```

```
scores = pd.DataFrame({
    "midterm": [92, 78, 85, 60, 71],
    "quiz_avg": [88, 74, 90, 58, 69],
})

pca = PCA(n_components=1)
combined_score = pca.fit_transform(scores)
print(combined_score.ravel().round(2))
# One column instead of two, capturing most of the shared information
```

At a larger scale, a PCA plot of gene-expression samples with thousands of original genes often reveals clustering by tissue type or disease subtype using just the first two or three principal components, well before any formal classification model is built — exactly the high-dimensional EDA strategy Part 2, Section 2.3 introduced. Students who want a fuller treatment of how PCA is derived and where it is used in practice may enjoy Jolliffe and Cadima’s accessible review [3].

1.4 Common Feature-Selection Approaches: Filter, Wrapper, and Embedded Methods

Three families of feature-selection method make different trade-offs between speed and how well they account for a specific downstream model.

Filter methods rank or select features using a statistical criterion computed independently of any downstream model — correlation with the outcome, mutual information, or a variance threshold — and are fast but ignore how features interact with a specific model.

Tip

A simple filter, by hand. Suppose three candidate features have the following correlation with house price: `square_feet` ($r = 0.71$), `lot_size` ($r = 0.58$), and `listing_agent_shoe_size` ($r = 0.02$). A simple filter rule — “keep any feature with $|r| > 0.3$ ” — would keep the first two and drop the third, without ever training a predictive model at all.

Wrapper methods evaluate subsets of features by actually training and validating a model on each candidate subset (forward selection, backward elimination, recursive feature elimination), which better captures feature interactions at substantially higher computational cost.

Embedded methods perform feature selection as an integral part of model training itself — an L1-regularized (Lasso) regression drives the coefficients of unhelpful features toward exactly zero, and tree-based models produce a feature-importance ranking as a natural byproduct of how they are built.

A commonly used filter criterion, **mutual information**, quantifies how much knowing one variable reduces uncertainty about another,

$$I(X; Y) = \sum_{x \in X} \sum_{y \in Y} P(x, y) \log \frac{P(x, y)}{P(x) P(y)},$$

and, unlike the Pearson correlation coefficient from Part 1, Section 1.6, is sensitive to nonlinear as well as linear relationships, which is why it is often preferred as a filter criterion when a relationship is suspected to be more complex than a straight line.

	Filter	Wrapper	Embedded
Speed	Fast	Slow (retrains per sub-set)	Moderate (one training pass)
Model-aware	No	Yes	Yes
Captures interactions	No	Yes	Partially
Leakage risk if misused	High (§1.5)	High (§1.5)	Lower, if regularization is tuned via cross-validation

Table 1: Trade-offs among the three families of feature-selection method.

```

from sklearn.feature_selection import SelectKBest, mutual_info_classif
from sklearn.ensemble import RandomForestClassifier

# Filter method: rank features by mutual information with the outcome
selector = SelectKBest(mutual_info_classif, k=10).fit(X_train, y_train)
top_features = X_train.columns[selector.get_support()]

# Embedded method: feature importance falls out of model training
rf = RandomForestClassifier(random_state=0).fit(X_train, y_train)
importances = pd.Series(rf.feature_importances_, index=X_train.columns)
print(importances.sort_values(ascending=False).head(10))

```

1.5 Feature Selection in Practice: Leakage and Bias

Feature selection is itself vulnerable to data leakage: selecting features by their correlation with the outcome computed on the *full* dataset, and only afterward splitting into training and test sets, lets information from the test set influence which features are chosen, inflating apparent performance in exactly the way Part 1, Section 2.7 warned against. **Selection bias** can also creep in through domain assumptions — systematically excluding features associated with a protected characteristic without checking whether a closely correlated proxy feature remains, which can silently reintroduce the same information the exclusion was meant to prevent.

Tip

Any feature-selection procedure that uses the outcome variable — filter methods based on correlation with the label, wrapper methods that validate against it, or embedded methods trained on it — must be performed *inside* the training fold of a cross-validation split, never on the full dataset beforehand. This single practice prevents the most common form of feature-selection leakage.

Real-World Case Study

A widely used benchmark for credit-card fraud detection contains transactions from European cardholders over a two-day period, in which confirmed fraudulent transactions make up only about 0.17% of the data — a degree of class imbalance that changes which feature-selection

strategy is appropriate [2]. With so few positive examples, a naive filter method based on raw correlation with the fraud label can be dominated by noise, since even an uninformative feature can appear weakly correlated with a rare class purely by chance — the same base-rate intuition from Part 1, Section 1.7, now working against a filter method rather than a screening test. The dataset’s original study instead focused on how the severe imbalance affects probability calibration and evaluation itself, a reminder that feature selection, model choice, and evaluation metric cannot be considered independently once class imbalance of this magnitude is present. In practice, feature selection on data like this is usually combined with resampling or class-weighting strategies and evaluated using precision-recall curves rather than raw accuracy, since a classifier that simply predicts “not fraud” for every transaction would already be correct more than 99.8% of the time.

Wrap-Up: Connecting Back

Feature selection and dimensionality reduction are not a final, separate step tacked onto the end of a KDD project — they are informed by, and feed back into, everything the rest of this unit covered. EDA (Part 1, and Part 2 Section 2) is usually what first surfaces which features look relevant, redundant, or irrelevant; the curse of dimensionality (Part 2, Section 3.4) is the concrete reason similarity-based methods (Part 2, Section 3) need a manageable feature set to work well; and both worked case studies in Part 2, Section 4 relied on feature selection to narrow dozens of raw fields down to a clinically interpretable set. Choosing carefully among filter, wrapper, and embedded methods — and always selecting features only inside a training fold, never on the full dataset first — is one of the concrete, learnable practices that Part 2’s discussion of common pitfalls (Section 6) built toward.

Discussion Questions

1. Section 1.1 distinguishes an irrelevant feature from a redundant one. Give one example of each from a domain you know well, and explain how you would check which category a suspicious feature falls into.
2. Section 1.3 shows PCA combining two highly correlated features (midterm score and quiz average) into one. Describe a different pair of features from a domain you know well that you would expect to be highly correlated, and explain what a single combined feature built from them would represent.
3. Section 1.4 argues that filter methods are fast but “model-blind,” while wrapper methods are model-aware but slow. Describe a situation where you would deliberately choose the faster, model-blind option even though it captures less.
4. Section 1.5 explains that feature selection must be performed inside, not outside, a cross-validation fold. Walk through, step by step, what would go wrong with a workflow that instead selected the top 10 features by correlation with the outcome on the *entire* dataset before ever splitting into train and test sets.

References

- [1] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [2] Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). Calibrating probability with undersampling for unbalanced classification. In *2015 IEEE Symposium Series on Computational Intelligence (SSCI)*, 159–166. <https://ieeexplore.ieee.org/document/7376606>
- [3] Jolliffe, I. T., & Cadima, J. (2016). Principal component analysis: A review and recent developments. *Philosophical Transactions of the Royal Society A*, 374(2065), 20150202. <https://doi.org/10.1098/rsta.2015.0202>
- [4] Hasan, K. (2026). *Knowledge Discovery and Data Mining*. Grand Valley State University. Available at: https://github.com/mdkamrulhasan/data_mining_kdd